

REXUS 38

STAR – D Experiment

Software Development
CONOPS – Concept of Operations Document



Version: 7.0
Issue date: 10/07/2026
Contributor: Alexandre Rollet
(Software engineer)

[PAGE INTENTIONALLY LEFT BLANK]

Table of Contents

1. **Introduction**
 - 1.1 Abstract
 - 1.2 Scope
 - 1.3 References
 - 1.4 Assumptions
2. **Mission overview relevant to software operations**
 - 2.1 Mission objectives
 - 2.2 Mission phases
 - 2.3 Success criteria
3. **Software operational concept**
 - 3.1 Concept of operations
 - 3.2 Software modes and state machine
 - 3.2.1 Modes
 - 3.2.2 Activation criteria
 - 3.2.3 State machine
 - 3.2.4 Operational scenario
 - 3.3 Flight computer interfaces
 - 3.4 Ground station software
 - 3.4.1 Purpose and scope
 - 3.4.2 Uplink commands
 - 3.4.3 Downlink handling
 - 3.4.4 Interface, operator display
4. **Data flow and management**
 - 4.1 Data list and types
 - 4.2 Data flow, processing and storage
 - 4.2.1 Data flow
 - 4.2.2 Storage control mechanism
 - 4.2.3 File/field format table
 - 4.3 Data output and telemetry
 - 4.3.1 Camera health definition
 - 4.3.2 Downlink, budget calculation
5. **Software constraints and requirements overview**
 - 5.1 Physical and environmental constraints
 - 5.2 Operational constraints
6. **Verification and validation overview**
 - 6.1 Verification concept
 - 6.2 Validation concept
7. **Abbreviations and notes**

1 – Introduction

1.1 Abstract

This document serves as a general description of the mission's software systems and development. It will give an overview of how the software will be designed and the requirements that will be enforced to ensure its robustness. The software will be explained through its role in the experiment, its functionality, expected operation times, and the framework used to validate its model.

1.2 Scope

This document is to be read as the project's design manual. Technical and project management elements will be presented to explain expected core functionalities and work development. General specifications that will need to be followed will be given. The document answers the question of *what must happen, when and why*. However, the document does not give specific technical details on software performance or reliable numerical data. Subsequent documents will explain precise requirements, software architecture and in-depth interface specifications.

1.3 References

- European Cooperation for Space Standardization (ECSS), ECSS-E-ST-40C Rev.1 – “Software Engineering”, 30 April 2025.

The above reference was used as a guideline for developing this document. The structure and content, however, do not strictly adhere to the official standard.

- RX_UserManual_v7-17_26Nov21

1.4 Assumptions

It is assumed that:

- all electronic components are operational or, at a minimum, meet the specified minimum performance requirements. External temperatures during the flight have no effect on electronic components

- no mechanical errors occur during the mission
- the sounding rocket performs a nominal flight
- telemetry loss does not affect onboard autonomy (however, details on this scenario will be given later in the scope of studying operational outcomes and risk management)

This allows potential software errors to be identified and evaluated within their respective domain.

2 – Mission overview relevant to software operations

2.1 Mission objectives

The software's objective is to enable semi-autonomous execution of the experiment for the whole duration of the mission. The software is responsible for data logging, temperature control, and telemetry transmission. Software-related mission objectives are as follows:

- The software autonomously and continuously monitors the temperature inside the experiment module, and enforces temperature corrections
- To collect, format and store camera data that records the microscope's view of the biology experiment
- To correctly format and transmit telemetry data with a fixed and limited packet size
- Acquire, manage, and preserve experimental data
- Provide experiment health monitoring through the telemetry downlink
- Detect and respond to off-nominal conditions
- Ensure modular and fault-tolerant architecture
- Coordinate experiment sequencing and operational modes

2.2 Mission phases

Phase	Explanation	Action(s)	Possible actions
Pre-launch, setup	Software upload to EM and startup	Startup and system test	Multiple test startups
Launch countdown	Countdown clock begins	SODS shortly before LO, receive telemetry, temperature control	Telecommands to start/stop SODS, activate RS, check system health
Launch	Rocket goes into flight	LO, higher frequency of metadata storage	
Ascent	Rocket ascends		
Motor burnout	Coast, microgravity	SOE on	
Descent	End of microgravity	SOE off, SODS off shortly before EM power cutoff	Lower frequency of metadata storage
Recovery	Recovery of data from storage card	Upload storage data into external computer	

Table. 1 Mission phases overview (simplified)

2.3 Success criteria

Software success is defined by its ability to function autonomously throughout the mission, to transition correctly between modes, to collect and preserve all crucial data from the experiment and sensors, and to continuously maintain the MFC at a strict temperature level. In this section, each success criterion is given verification evidence that is traceable and a criticality factor.

ID	Success criterion	Operational phase	Verification evidence	Criticality
SC-01	Software can be uploaded to hardware without errors	Setup	No crash, successful upload observed on electronics LEDs	Critical
SC-02	Software boots nominally after power-on	Setup	Boot logs, electronics LED status	High
SC-03	Transition to flight mode during launch detection/uplink signal	Countdown, launch	Telemetry logs, downlink observation	High
SC-04	Software manages to maintain stable module temperature	Ground tests, flight	Sensor output, downlink observation	Critical
SC-05	Telemetry is correctly formatted and sent	All phases (exc. RS)	Telemetry logs, no bandwidth issues	Medium
SC-06	Experiment begins at uplink signal reception	Microgravity	Telemetry logs, microscope LEDs	Critical
SC-07	No unrecoverable errors	All phases	Error logs	Medium
SC-08	No error shall crash the software	All phases	Error logs, overall system health	Critical
SC-09	Microscope imaging is correctly collected throughout	All flight phases	Telemetry logs, microscope status	High
SC-10	An unexpected crash shall not corrupt previously collected data	All phases	Error logs, check stored data	High
SC-11	Software must correctly receive LO/SOE/SODS	All phases	Experiment status, logged files	Critical

Table. 2 Success criteria list with verifications and criticality

3 – Software operational concept

3.1 Concept of operations

This segment explains how the software will act throughout the mission. The software will be developed in a way that maximizes autonomous functionality. Key characteristics of the software will be:

- Three critical ground commands will be sent during the mission via the RXSM to do the following operations: LO (at lift-off, start onboard clock), SOE (introduce biological particles and medium to microscope observation area), SODS (start recording and storing all data).
- The software will have a deterministic execution regarding error handling and the startup. It will have to transition to the appropriate mode depending on the onboard clock, which starts running at LO.
- The software is set to be fault recoverable but not fault tolerant. All potential errors will, to the best extent possible, be targeted during development, so that the least number of errors occur during flight. The code will have to be able to recover from any faults that occur.

The following schematic shows a simplified overview of the software architecture:

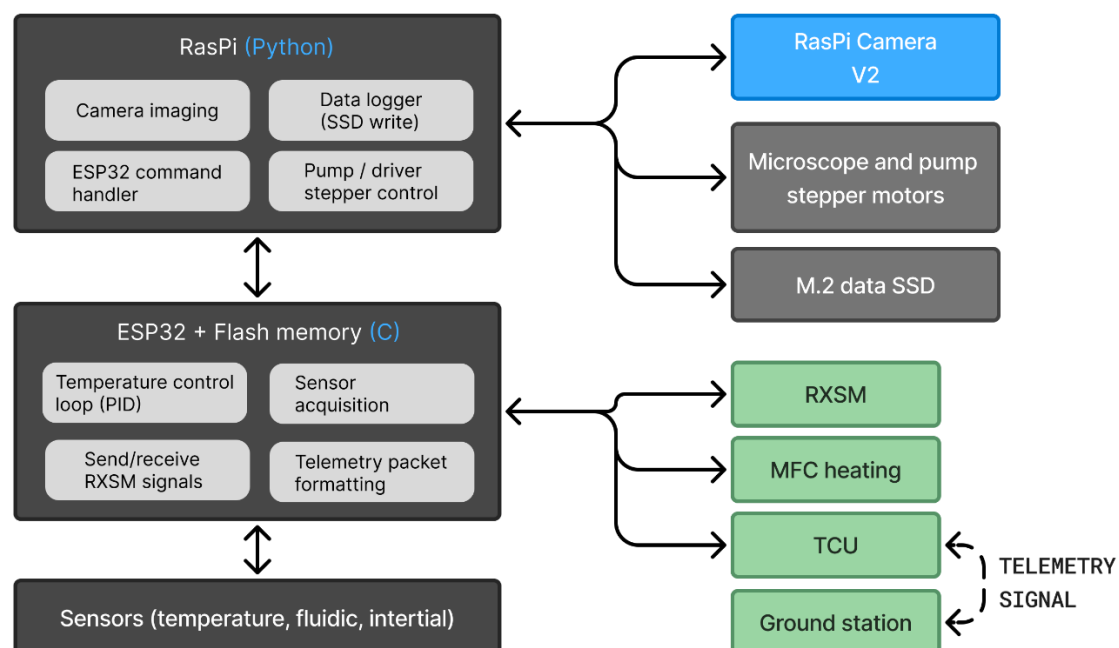


Fig. 1 Software architecture overview

The ESP32 will remain in the experiment design alongside the RasPi to guarantee deterministic timing for the closed-loop temperature control (PID) and actuator drive signals (heater, stepper motors), functions tied to *critical* success criterion SC-04 that cannot be reliably guaranteed by the RasPi's operating system, particularly given the RasPi's own CPU-intensive imaging workload. The ESP-RasPi interface is limited to a single, minimal UART link.

3.2 Software modes and state machine

3.2.1 Modes

Mode	Abbreviation	Use/Scope
Startup	SU	Boot experience, software begins running
Test	T	Verify system health
Normal ground	NG	Software is functional on the ground
Normal flight	NF	Software is functional during flight
Normal experiment	NE	Software is functional and experiment is ongoing

Table 3 Definition of operational modes

The fallback mode, which previously served as a graceful degradation mode in the case of persistent major malfunctions, was removed due to it being overly complex in the scope of the mission. However, several core ideas will be maintained, such as:

- Using simple try/catch logic
- Wrapping sensor reads. Malfunctions are simply logged and skipped, instead of cascading
- Data writes are small, and are stocked in fixed-size files (see section 4.2)
- Error flags are used as indications and logged

3.2.2 Activation criteria

Mode	Activation
SU	Once the EM is connected to electrical power
T	By telecommand from ground station
NG	Once the startup phase is complete
NF	After the LO signal is received, & after SOE off
NE	After the SOE signal is received, turns off when SOE is deactivated

Table 4 Activation criteria to transition into a given operational mode

3.2.3 State machine

- **SU**: During the module startup, the main software functions begin working throughout the hardware.
- **T**: If a *test* telecommand is sent from the ground station, a series of tests begin to ensure that the system's health is flight-worthy. The tests include:
 - Verify that hardware is functioning (CPU, accessible memory, peripherals and interfaces).
 - Verify that sensors are active and that they transmit coherent data.
 - Verify that there is power running in the system and that the system clock can run correctly (important for mission timestamps).
 - Verify that there is available storage space, ability to upload data into storage (there is a very large amount of storage, but a corrupted data card could quickly invalidate this).
- **NG**: After and if the tests are validated, the normal ground mode starts. No major operations occur; the system is GO for launch. Temperature control is active, and data storage starts at SODS shortly before launch.
- **NF**: Activated at the reception of the LO signal. Data logging frequency is increased, and temperature control continues. Later, after the SOE signal is turned off, NF mode is used for the remainder of the flight and descent.
- **NE**: Activated once the SOE signal is activated. This starts the experiment operations and continues with the previous tasks. Only operational during the experiment phase.

3.2.4 Operational scenario

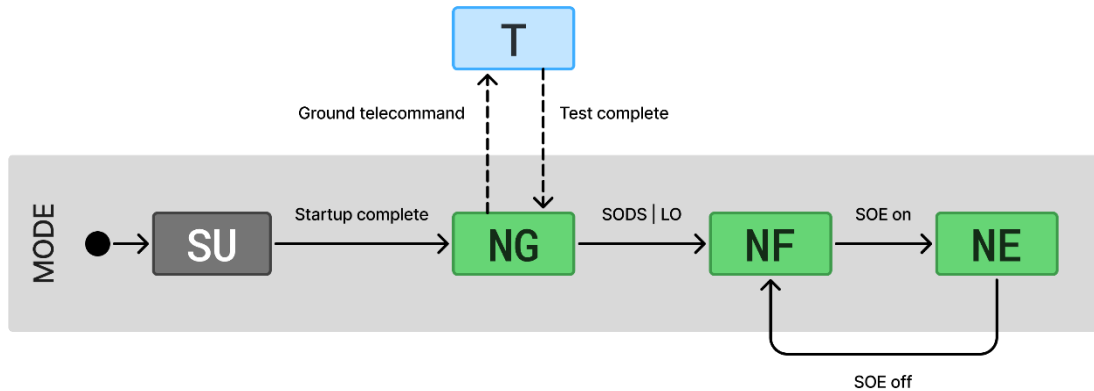


Fig. 2 State transitions

Near the end of the flight, SODS is turned off and the system awaits power cutoff at T+600s. All data is stored in memory by this point, and the EM runs the NF mode until cutoff.

There is no need for a Shutdown procedure before RS, or before power cutoff at the end of the flight.

After the flight, data will be recovered by extracting it from the storage cards and from the flight computers. All data recovery can be done offline. The recovered videos will show the results of the experiment. All the other metadata can be used to design graphs that can be correlated to experiment phases and performance.

3.3 Flight computer interfaces

Both flight computers (ESP and RasPi) are connected by a UART interface. SPI will serve as the interface for most components, including both BME temperature sensors, the sensitive PT100 temperature sensor, the LSM6DSM inertial sensor, and the ABP pressure sensor. Due to its design, the SLF35 liquid flow sensor will have an I²C interface. As shown in *Fig. 3*, a UART interface will communicate with the TCU, and a RS-422 will communicate with the RXSM.

3.4 Ground station software

3.4.1 Purpose and scope

The ground station enables the following:

- Continuously monitor all EM parameters, especially temperature values that are critical for the experiment
- Before LO, commands will allow the ground team to activate a test mode, wipe SSD data, and override the MFC heater if needed
- Log a backup of the received telemetry onto the ground station computer

Since no imagery is received by the ground station, and no TV channels are used, it will serve as a control display to see if all parameters and undergoing processes are nominal. This will be sufficient for the ground team to consider that the experiment is unfolding correctly. A flag that confirms that the stepper motor has been activated will be nested on the condition that the SOE signal has been received. This allows the ground station to confirm that bacterial particles have been injected onto the MFC.

3.4.2 Uplink commands

The ground station will be able to send telecommands before LO through uplink to do the following actions:

Uplink command	Use	Logic
TEST	Activates the test mode	Test mode can be run at any time and as many times as desired
WIPE	Wipes data from the M.2 data SSD	In case of prolonged holds, it ensures there is sufficient storage before launch
HEAT	Overrides the MFC heater PID loop and briefly warms the MFC	If MFC temperature becomes critically low for any reason, it can be reheated manually to keep biological samples alive

Table 5 Uplink commands and scope

The commands above will all require confirmation from the ground team before being executed to avoid any accidental actions.

3.4.3 Downlink handling

The ground station directly decodes the received downlink bytes following the same structure defined by the telemetry packet format. Packet sequence gaps are flagged, but lost packets do not interrupt the reception of subsequent packets.

Telemetry will provide a live indication of any errors that may arise. A `uint32_t` variable will serve as an error flag, sent in each telemetry packet. Possible errors are predefined using a `typedef enum` structure, and if one arises, the corresponding bit number of the flag will be flipped to 1. Once decoded by the ground station, the presence or absence of errors will be visible. This is more efficient than sending an entire error message through downlink.

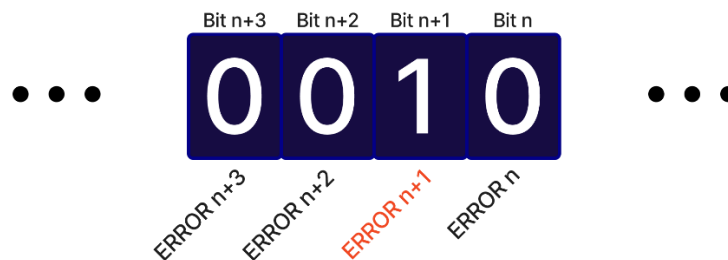


Fig. 3 Error flag representing an error indication based on bit status

3.4.4 Interface, operator display

The UI will be optimized to visibly present the most crucial variables for the ground team to follow. Some variables such as temperature will be used to construct graphs, that will help track its evolution.

4 – Data flow and management

4.1 Data list and types

This section describes all the data that is handled by the experiment and explains its purpose, and when said data is to be collected/received/sent.

Data ID	Source	Type	Category	Purpose	Mission phase
DA-01	ESP-32 external flash memory	Acquired data	Various, temporary storage	Allows ESP-32 to handle more data if needed	All phases
DA-02	BME sensors	Acquired data	Environmental	Characterize experiment environment	All phases
DA-03	Gyroscope (LSM6DSM)	Acquired data	Inertial	Motion data	Launch, ascent, microgravity
DA-04	PT100 temperature sensor	Acquired data	Environmental	Closed-loop temperature control	All phases
DA-05	RasPi Camera V2	Acquired data	Image	Visualization of experiment	Throughout the flight
DA-06	M.2 storage	Acquired data	Metadata, imaging	Mission logs and microscope imaging	All phases
DA-07	RXSM	Acquired data	Experiment signals (LO, SOE, SODS)	Initiate respective mission phases	Countdown, LO, ascent and microgravity
DA-08	TCU to ground station	Transmitted data	Systems data	Flight status downlink	All phases
DA-09	ABP sensor	Acquired data	Biology	Fluid pressure	Experiment
DA-10	SLF35 sensor	Acquired data	Biology	Fluid flow	Experiment
DA-11	ESP32 to TCU	Transmitted data	Telemetry packet	Downlink data	All phases
DA-12	Ground station to TCU	Command	Uplink commands	Wipe data, activate test mode, override heater	Pre-launch

DA-13	Microscope and pump stepper motors	Command	Activate drivers	Inject particles and focus microscope	Experiment
-------	------------------------------------	---------	------------------	---------------------------------------	------------

Table 6 Data sources, type and purpose

4.2 Data flow, processing and storage

4.2.1 Data flow

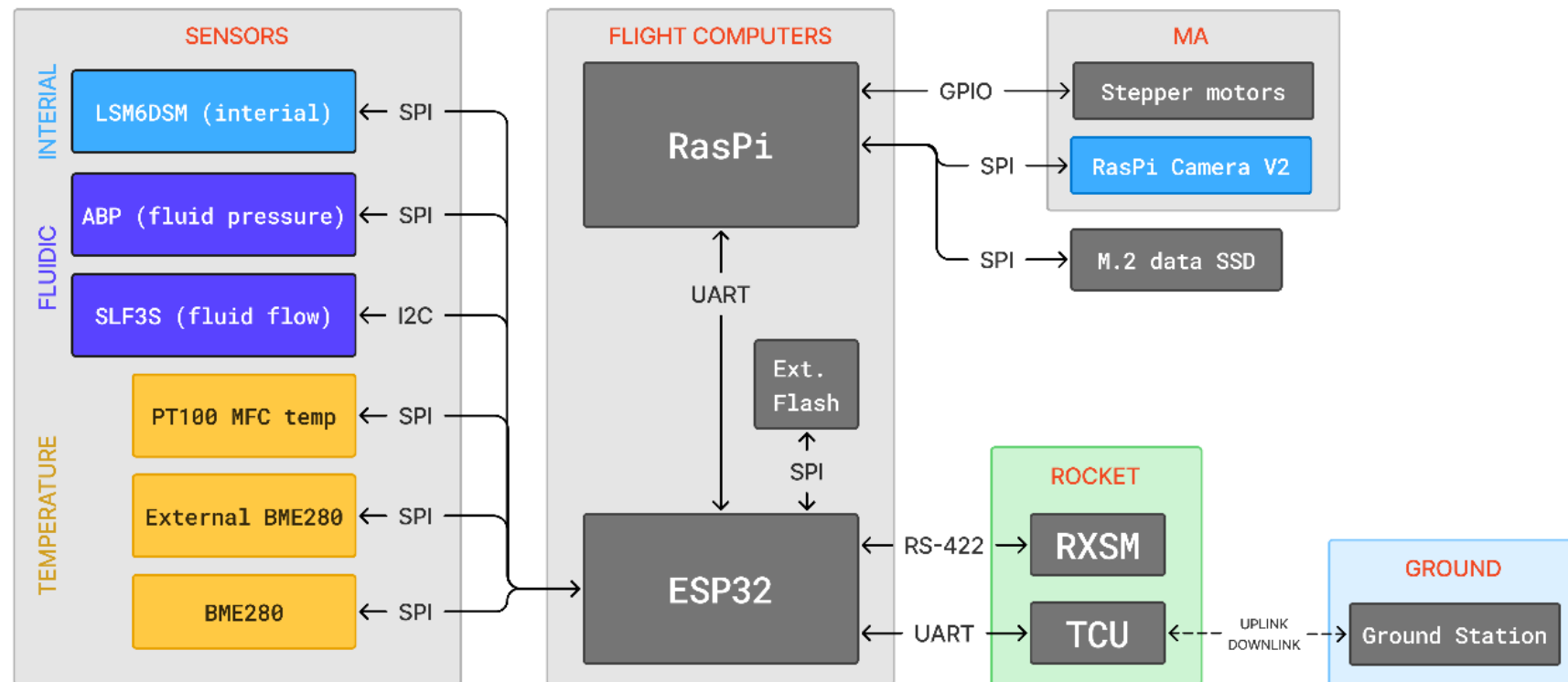


Fig. 4 System interfaces visual diagram

The interface that connects the ESP32 to the RasPi has been changed to a UART interface.

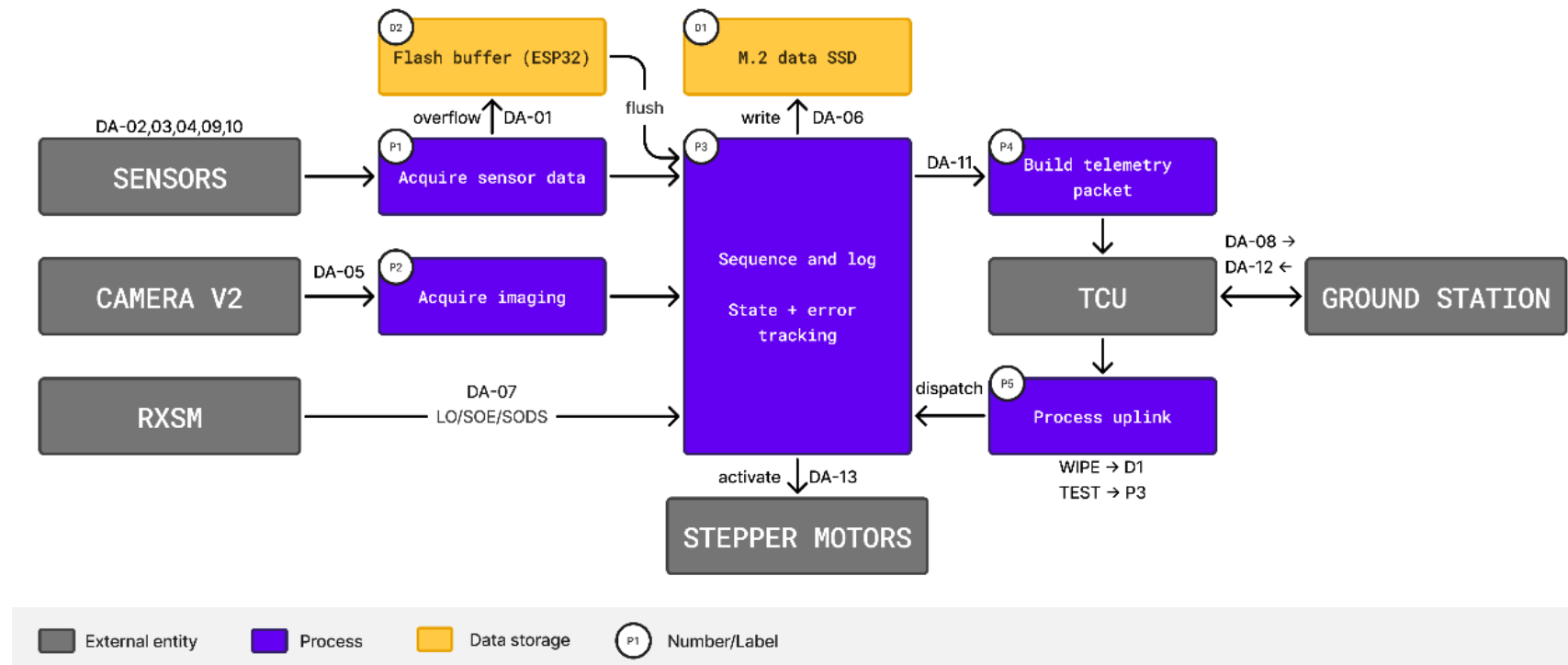


Fig. 5 Data flow diagram

4.2.2 Storage control mechanism

- Write trigger: All logged data is read from an in-memory table that is continuously updated as data arrives. This is also the case for structuring telemetry packets (fig. 5, P4), which are not assembled from stored data. This avoids risks linked with reading files during the mission.
- Category separation: Data is logged into separate files, each with its own category (see table 6).
- File rotation policy: File rotation size is smaller during the experiment phase when SOE is active, since experiment data is critical and its loss should be minimized. File rotation validates criterion SC-10 from table 2.
- Naming convention: Each file/frame name includes category, sequence number, and a timestamp anchor so files are unambiguously orderable and joinable post-flight even if some are missing.
- Fault containment: Identically to sensor reads, write malfunctions are simply logged and skipped rather than propagating.

4.2.3 File/field format table

This section specifies data storage formats and frequencies.

Category	Fields	Format	Frequency/sample rate	Rotation (new file every N-lines/frames)
		<i>SOE on / SOE off</i>	<i>SOE on / SOE off</i>	<i>SOE on / SOE off</i>
System state metadata	Timestamp, mode, error flag, SSD free space	.csv	10 Hz (unified row logging cadence)	1000/200
Sensor metadata	Timestamp, BME x2, PT100, ABP, SLF35, LSM6DSM	.csv	10 Hz (native sample rate)	1000/200
Telemetry output	All metadata and sensor values	.csv (mirrors downlink packet)	1 Hz (subsampling slice of data above, not independently sampled)	100/20

Imaging	Frame timestamp, resolution	.mp4 / JPEG	60 fps / 4 fps	6000/400
Uplink commands	Timestamp, command ID	.csv	On demand (on ground only)	N/A. One file sufficient
Actuator commands	Timestamp, target, confirmation flag	.csv	Event-triggered	N/A. One file sufficient

Table 7 Data storage formats and logging frequency

File size rotation is set to a reasonable cycle to prevent data loss if one file is corrupted during logging. Depending on the different logging frequencies, the size rotation is adjusted between data types to align file cycles (1000 lines at 10 Hz is equivalent in time to 400 frames at 4 fps, etc.).

.mp4 video will be recorded at 720p60 and encoded with H.264. For a 2–3-minute experiment window, this equates to a 120 MB / 180 MB video file, which poses no data storage issue.

4.3 Data output and telemetry

4.3.1 Camera health definition

To check if the camera is functioning correctly, which is a non-negotiable criterion for experiment results, the software will run two checks to confirm that imaging is in progress:

- The RasPi receives timestamped pings from the camera and verifies that the two consecutive timestamps are increasing. This proves that the camera sensor is alive
- The RasPi checks that the size of the total imaging write has increased at every code iteration. This confirms that data is being written into storage.

```
advancing = sensor_ts > last_sensor_ts
last_sensor_ts = sensor_ts

write_ok = size_now > last_written_size
last_written_size = size_now

if not advancing:
    return CAM_STALLED
if not write_ok:
    return CAM_WRITE_FAIL
```

Fig. 6 Camera health software verifications

Each camera flag is defined in a `typedef enum` and sent as a single byte through downlink.

4.3.2 Downlink, budget calculation

The telemetry downlink will send all available data to the ground station, except for imaging which requires a very high budget and image loss is highly possible. All telemetry downlink data is identically stored in the M.2 SSD.

Each telemetry packet contains the following items:

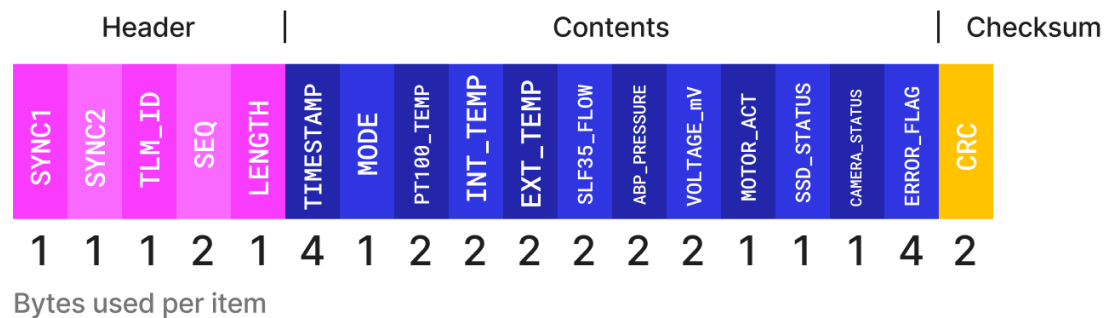


Fig. 7 Telemetry downlink protocol

Each packet has a length of 32 bytes, which at a 1 Hz downlink frequency gives a downlink budget of 256 bits/s. By adding a margin of 25% in case additional fields are added, this gives a downlink budget of 320 bits/s, or 0,320 kbits/s. This value is well below the TCU’s downlink limits.

5 – Software constraints and requirements overview

5.1 Physical and environmental constraints

External constraints imposed by the hardware and the mission environment will bring some limits to the system's capabilities. The software is designed to operate well within the processing limits of the flight computers, to ensure lightweight execution of the code. The software will account for transient faults and will need to detect them to contain the issue and to autonomously switch to a recovery state.

Vibrations, quick temperature variations and radiation effects could have an impact on the system, so tolerance levels will have to be set. Since there is no in-flight access to any onboard systems, autonomy will have to be at the center of developing how the software must act overall.

5.2 Operational constraints

The software operates under the constraint of a semi-autonomous mission profile. LO, SOE and SODS signals are sent from the RXSM, but all the other operational behavior is pre-defined prior to launch, and the software must run reliably within a fixed mission timeline.

6 – Verification and validation overview

6.1 Verification concept

Software verification will be performed using unit tests during code development, integration tests in a high-fidelity simulation environment, and formal code inspections.

6.2 Validation concept

Validation will be performed by executing representative mission scenarios in a controlled simulation environment, measuring performance against defined mission success criteria, and obtaining operational approval from the REXUS/BEXUS team.

7 – Abbreviations and notes

EM	Experiment module
LO	Lift-off (signal)
SOE	Start of experiment (signal)
SODS	Start of data storage (signal)
TCU	Telemetry central unit
T	Time before and after launch noted with + or –
RXSM	REXUS Service module
RS	Radio silence
MFC	Microfluidic chip

[END OF DOCUMENT]